

J3

Fichiers, formats et robustesse

Lire les vrais fichiers du dépôt — CSV et JSON —, encaisser les données sales avec les exceptions, organiser son code en paquet.

05-fichiers-et-formats

06-erreurs-et-exceptions

07-modules-et-organisation

Objectifs de la journée



Lire et écrire CSV / JSON

DictReader, DictWriter, json.load — avec pathlib et UTF-8.



Encaisser les données sales

try/except : quatre formats de date, montants vides ou négatifs.



Organiser en paquet

Le paquet fiscalite/ : dates et pénalités réutilisables partout.



Comprendre uv

pyproject.toml, uv sync, uv add — d'où viennent les bibliothèques.

LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

Objets & pandas

J5

Intégration & projets

Fil rouge : normaliser les dates des déclarations.

Ouvrir un fichier, proprement



pathlib.Path, pas de chaînes collées

DATA / "regions.csv" — portable Windows et Linux.



with ferme toujours

Même en cas d'erreur au milieu — plus de fichiers zombies.



encoding="utf-8", toujours

Sinon les accents cassent selon le poste. Non négociable.



Chemins relatifs au script

Path(__file__).parent : le script marche d'où qu'on le lance.



tp_lecture.py

```
from pathlib import Path

DATA = Path(__file__).parent \
    .parent / "00-data" / "sortie"

with open(DATA / "regions.csv",
          encoding="utf-8") as f:
    for ligne in f:
        ...

# fermé automatiquement ici
```

CSV : chaque ligne devient un dictionnaire



csv.DictReader

La 1re ligne donne les clés : ligne["nif"], ligne["code_region"].



Tout arrive en str

Les montants se convertissent avec int() — et ça peut planter (cf. module 06).



DictWriter pour produire

writeheader() puis writerow(dict) — l'extrait Conakry du TP.



newline="" en écriture

Sinon Windows double les sauts de ligne.



comptage par région

```
import csv

with open(DATA / "contribuables.csv",
          encoding="utf-8") as f:
    for ligne in csv.DictReader(f):
        code = ligne["code_region"]
        compte[code] = \
            compte.get(code, 0) + 1
```

Conakry : 2513 Kankan : 2461 ...

JSON : des structures imbriquées



json.load → dicts et listes

Le fichier devient exactement les structures du J2.



On navigue par clés

`d["identite"]["ville"]`, `d["etablissements"][0]["actif"]`.



any() et all()

« au moins un établissement actif » tient en une ligne.



Cas réel du dépôt

`dossiers_fiscaux.json` : identité + établissements + historique.



dossiers imbriqués

```
import json

with open(DATA / "dossiers_fiscaux.json",
          encoding="utf-8") as f:
    dossiers = json.load(f)

n = sum(
    1 for d in dossiers
    if any(e["actif"]
          for e in d["etablissements"]))
```

Exceptions : le code qui ne panique pas



Lire un traceback de bas en haut

Dernière ligne : QUOI. Remontée : OÙ. Ne pas paniquer.



Attraper PRÉCIS

except ValueError — jamais except: nu, qui masque tout.



raise pour vos propres règles

Montant vide ou négatif → ValueError avec un message clair.



Écarter EN COMPTANT

Une ligne rejetée en silence est une ligne perdue.



tp_dates_sales.py

```
def lire_montant(txt: str) -> int:
    if txt.strip() == "":
        raise ValueError("vide")
    m = int(txt)    # peut lever
    if m < 0:
        raise ValueError(f"négatif")
    return m

try:    lire_montant(ligne["ca"])
except ValueError:    invalides += 1
```

Quatre formats de date dans UN fichier

2025-03-14

14/03/2025

14-3-2025

20250314

date_depot dans declarations.csv — exactement comme dans les fichiers administratifs réels



Essayer chaque format

strptime dans un try/except ValueError, format après format.



Renvoyer None si tout échoue

L'appelant décide : écarter, corriger, signaler.



le normalisateur

```
FORMATS = ["%Y-%m-%d", "%d/%m/%Y", ...]

def normaliser(brute: str):
    for fmt in FORMATS:
        try:
            d =.strptime(brute, fmt)
            return d.strftime("%Y-%m-%d")
        except ValueError:
            continue
    return None
```

Du fichier unique au paquet réutilisable

```
07-modules-et-organisation/
├─ fiscalite/           # le paquet
│   ├── __init__.py
│   ├── dates.py        # normaliser()
│   └─ penalites.py     # penalite()
└─ tp_utiliser_paquet.py
```

```
from fiscalite import normaliser, penalite
penalite(8_100_000, 45) # → 972000
```



Un module = une responsabilité

dates.py ne parle que de dates ; penalites.py que de pénalités.



__init__.py expose l'essentiel

from fiscalite import penalite — l'utilisateur ignore l'interne.



uv gère les bibliothèques

uv add pandas écrit pyproject.toml ; uv sync reproduit partout.

Les vraies données entrent en scène

- 1 **tp_lecture.py** — référentiel, comptage par région, extrait Conakry, JSON
- 2 **tp_dates_sales.py** — normaliser 4 formats + compter les montants invalides
- 3 **fiscalite/penalites.py** — transférer vos fonctions du J2 dans le paquet
- 4 **tp_utiliser_paquet.py** — importer et utiliser votre propre paquet



Fichiers du jour

```
05-fichiers-et-formats/  
06-erreurs-et-exceptions/  
07-modules-et-organisation/  
    fiscalite/
```



Régénérez les données si besoin : `cd 00-data
&& python generate_data.py --auto`

À retenir

- ✓ **with + utf-8 + pathlib** — le trio d'ouverture de fichier — désormais automatique.
- ✓ **DictReader** — un CSV se parcourt comme une liste de dictionnaires.
- ✓ **except PRÉCIS** — on attrape ValueError, pas « tout » — et on compte ce qu'on écarte.
- ✓ **normaliser()** — les 3 000 dates du fichier passent, zéro plantage.
- ✓ **Le paquet fiscalite** — votre code du J2, devenu réutilisable dans tout le dépôt.



Demain — J4 : objets et pandas

Modéliser Contribuable et Declaration en dataclasses, puis refaire tout le nettoyage du jour en dix fois moins de lignes avec pandas. Et : lancement du projet final.